

(패키지 의존성) 왜 비슷한 패키지가 여러 개 있지?

초보자가 가장 헷갈리는 부분이 바로 "왜 비슷한 패키지가 여러 개 있지?"라는 점 입니다. 결론부터 말하면 `xlrd` 와 `openpyxl` 은 경쟁 관계라기보다 시대에 따라 역할이 바뀐 패키지라고 이해하면 됩니다.

1. 누가 먼저 나왔나?

패키지	최초 공개	목적
<code>xlrd</code>	2005년경	Excel 읽기(.xls)
<code>xlwt</code>	2005년경	Excel 쓰기(.xls)
<code>openpyxl</code>	2010년경	Excel(.xlsx) 읽기·쓰기

xlrd(구세대) → openpyxl(신세대) 순으로 등장!

2. 왜 openpyxl이 새로 만들어졌나?

1) 엑셀 파일 형식이 바뀜

예전 Excel(2003 이전)

.xls

↓

Excel 2007부터

.xlsx

로 바뀜 `.xlsx` 는 내부적으로

```
ZIP
├─ sheet1.xml
├─ styles.xml
├─ workbook.xml
└─ ...
```

처럼 XML 파일들의 묶음

기존 `xlrd` 는 옛날 `.xls` 형식을 다루기 위해 만들어졌기 때문에 새로운 형식을 충분히 지원하기 어려웠습니다.

그래서

|"xlsx 전용으로 제대로 만들자."

라는 생각으로 `openpyxl` 이 등장했습니다.

2) 기능 차이

`openpyxl` 은

- 셀 스타일
- 색상
- 폰트
- 수식
- 차트
- 이미지
- 병합 셀

등 Excel의 거의 모든 기능을 다룰 수 있음

반면 `xlrd` 는

| "데이터만 읽는" 목적에 가까움

3. 왜 지금은 xlrd를 거의 안 쓰나?

가장 큰 사건은 2020년 `xlrd 2.0` 부터

| `.xlsx` 지원을 제거 함

즉 예전에는

```
import xlrd

book = xlrd.open_workbook("test.xlsx")
```

가능했는데, 지금은

```
XLRDError:
Excel xlsx file; not supported
```

오류가 납니다. 그래서 현재는

파일	추천
.xls	xlrd
.xlsx	openpyxl

이렇게 구분됩니다.

4. 그런데 Pandas는 왜 그냥 읽히지?

예를 들어

```
import pandas as pd
```

```
df=pd.read_excel("data.xlsx")
```

이 코드에는 `openpyxl` 이 보이지 않습니다. 하지만 내부에서는

```
graph TD
    pandas --> openpyxl
    openpyxl --> excel[Excel 파일]
```

이렇게 동작합니다. 즉

Pandas는 직접 Excel을 읽는 것이 아니라 적절한 엔진(engine)을 사용합니다.

대표적인 엔진은

- openpyxl
- xlrd
- odf
- pyxlsb

등이 있습니다.

5. 이것이 바로 "패키지 의존성"

예를 들어

```
graph TD
    program[내 프로그램] --> Pandas
    Pandas --> openpyxl
    openpyxl --> Python
```

이런 구조인거죠. 즉

Pandas는

| "Excel은 내가 읽는 게 아니라 openpyxl에게 부탁할게."

라고 생각하면 됩니다. 이것을

| **의존성(Dependency)**

이라고 합니다.

6. 초보자는 패키지를 어떻게 이해하면 좋을까?

컴퓨터를 회사로 생각하면 가장 이해하기 쉽습니다.

```
사장
|
▼
내 코드

직원
|
▼
Pandas

전문가
|
▼
openpyxl

현장
|
▼
Excel 파일
```

- 나는 Pandas에게 일을 시킴
- Pandas는 Excel 전문가(openpyxl)에게 일을 맡김
- openpyxl이 실제 파일을 처리함

즉

패키지는 서로 일을 분담하는 협업 구조라고 이해하면 된다.

7. 파이썬 생태계는 대부분 이런 구조

예를 들어 머신러닝도 마찬가지입니다.

```
scikit-learn
|
▼
numpy
|
▼
BLAS/LAPACK
```

또는

```
matplotlib
|
▼
numpy
```

또는

```
pandas
|
▼
numpy
```

처럼 대부분의 라이브러리는 더 낮은 수준의 전문 라이브러리에 의존합니다.

8. 초보자가 가져야 할 관점

처음에는 다음 3단계만 기억하세요.

단계	이해할 내용
1단계	필요한 기능을 제공하는 패키지가 무엇인지 이해하기 (예: pandas는 데이터 분석)
2단계	그 패키지가 내부적으로 다른 패키지에 의존할 수 있다는 사실 이해하기
3단계	오류가 발생하면 "내 코드 → 사용하는 패키지 → 그 패키지의 의존성" 순서로 원인을 찾아 보기

이 관점을 익히면 `openpyxl` 과 `xlrd` 뿐 아니라 `numpy` , `pandas` , `matplotlib` , `scikit-learn` 등 대부분의 파이썬 라이브러리 관계도 자연스럽게 이해할 수 있습니다.